

C Advanced Essentials

Structures

```
1 struct STUDENT {  
2     char name[26];  
3     float time;  
4     int recent_chapter;  
5 };
```

the declaration of the structure always starts with the keyword **struct**;
a **struct tag** is placed after the keyword (STUDENT in the case); it's the name of the structure itself; there's a widely accepted custom of composing structure tags with capital letters only to distinguish them from ordinary variables; here comes the opening curly bracket – a sign that the field declaration begins at this point; our structure has three fields: the first is a 26-element array of type char and is called name; the second is a float and is called time, and the third is an int and is called **recent_chapter**;
the declaration ends with the closing curly bracket followed by a semicolon

```
struct STUDENT stdnt;
```

This declaration sets up a variable (a **structured variable**) named stdnt. The variable **stdnt** is of type struct **STUDENT**. We know that this variable consists of three named fields, but we don't yet know how to access them.
As the "C" language offers a specialized indexing operator **[]** for arrays, we also have the selection operator, designed for structures and denoted as a single character **.** (dot)

Windows

C:\directory\file

Linux

/directory/files

As you can see, systems derived from Unix **don't use the disk drive letter** (e.g. C:) and all directories “grow” from one root directory called /, while Windows systems recognize the root directory as <drive_letter>:\

This means that these two strings:

- **ThisIsTheNameOfTheFile**, and
- **thisisthenameofthefile**,

describe two different files in **Unix** systems, but are the same name of just one file in **Windows** systems.

Any program written in the “C” language (although not only in “C”, because this convention applies to virtually all programming languages) does not communicate with the files directly, but through an abstract **entity called a stream**. The programmer, having a rich set of functions defined mainly in the header file **stdio.h**, can perform certain operations on the stream which affect the real files using mechanisms contained in the **operating system kernel**

The opening of the stream is performed by a function of the following prototype:

```
FILE *fopen(char *filename, char *openmode);
```

the opening must be the very first operation performed on the stream (there's a triple exception to this rule – more details soon)

r open mode: **read**

w open mode: **write**

a open mode: **append**

r+ open mode: **read and update**

w+ open mode: **write and update**

If the letter **t** is at the end of the mode string, it means that the stream is to be opened in text mode

If the letter **b** is at the end of the mode string, it means that the stream is to be opened in binary mode

- we've declared a variable of type **FILE ***; we'll use it to represent a file associated with the stream named file;
- we use the **fopen** function to try to open the specified file (note the doubled backslashes);
- the open mode is defined as **text to read**;
- we get a pointer from the **fopen** function and we assign it to the file variable; then we check the variable to see if it's equal to **NULL**; the confirmation of this comparison proves that the file could not be opened

stdin (standard input);

the **stdin** stream is normally associated with the keyboard, pre-opened for reading and regarded as the primary data source of running programs;

the **scanf** function reads the data from **stdin** by default

stdout (standard output);

the **stdout** stream is normally associated with the **screen**, pre-opened for **writing**, regarded as the primary target for outputting data by the running program

stderr (standard error output);

the **stderr** stream is normally associated with the **screen**, pre-opened for **writing**, regarded as the primary place where the running program should send information on the errors encountered during its work

The last operation performed on a stream (this doesn't include the **stdin**, **stdout**, and **stderr** streams which don't require it) should be the **closing**. This action is performed by a function with the following prototype:

```
int fclose(FILE *stream);
```

The definition of the **errno** variable (the name comes from the phrase **error number**) is located in the **errno.h** header file and it look as follows:

```
extern int errno;
```

EACCES	Permission denied
EBADF	Bad file number
EEXIST	File exists
EFBIG	File too large
EISDIR	Is a directory
EMFILE	Too many open files
ENOENT	No such file or directory
ENOSPC	No space left on device

```

1 FILE *file = fopen("c:\\file.txt","rt");
2 if(file == NULL) {
3     switch(errno) {
4         case ENOENT: printf("The file doesn't exist\n");
5                     break;
6         case EMFILE: printf("You've opened too many files\n");
7                     break;
8         default:    printf("The error number is %d\n",errno);
9     }
10 }

```

Fortunately, there's a function that can dramatically **simplify** error-handling code. It's called **strerror**, its prototype is located in the **string.h** file and it looks like this:

```
char *strerror (int errnum);
```

Reading functions:

fgetc()	reading one character from the stream
fgets()	reading one string from the stream
fread()	reading bytes from the stream
fscanf()	formatted reading from the stream

Writing functions:

fputc()	writing one character to the stream
fputs()	writing a string to the stream
fwrite()	writing bytes to the stream
fprintf()	formatted writing to the stream

The preprocessor looks for a file named **filename**, but if you enclose the name inside quotes, it searches for that file in the same directory as the file containing the directive; otherwise it searches for the file in the compiler's default directory; Unix systems will usually **use /usr/include** for this purpose;

if the file is not found, the preprocessor reports an error; otherwise, the **#include** directive is replaced with the content of the included file;

the included file **may contain** an **#include** directive, too; in addition, a particular compiler may impose a certain limit on the number of nested inclusions;

The **#define** directive may take one of the following three forms shown in the editor:

```

1 #define identifier text
2
3 #define identifier(parameter_list) text
4
5 #define identifier

```

The `#undef` directive **cancels** the effects of the selected `#define` directive and takes the following form:

```
#undef identifier
```

`__LINE__` identifier it is replaced by an **integer literal equal to the line number**

`__FILE__` identifier is always replaced by a string literal containing the **name of the source file**

`__DATE__` identifier is always replaced by a string literal containing **text denoting the day** the source file was compiled

`__TIME__` identifier is always replaced by a string literal containing **text denoting the time** (hours, minutes, seconds) the source file was compiled

`__STDC__` identifier (as in Standard C) is defined if and only if the following statement is true

The `#` operator (don't confuse it with the `#` at the beginning of the directives) is a prefix unary operator which is used in the way we've shown here:

```
#argument
```

Note that you can't do this in any other way, because a macro similar to the following:

```
#define QUOTE(X) "X"
```

will not work as expected. The preprocessor doesn't look inside the string literals. Here's the correct solution:

```
#define QUOTE(X) #X
```

The `##` operator is a binary operator requiring two arguments, and is used in the following way:

```
ARGUMENT1 ## ARGUMENT2
```

Note that you can't do this in any other way because a macro similar to the following:

```
#define GLUE (X,Y) XY
```

will not work. The preprocessor recognizes the `XY` identifier, not `X` and `Y`.

Here's the correct solution:

```
#define GLUE(X,Y) X##Y
```

An internal variable declared as an extern

```
extern int amount;
```

A variable declared as a local variable in a module (outside any block) causes the compiler to assume that a variable with the same name is defined in another module. It also means that other variables declared as `extern`, and with the same name, will be treated as a reference to one and the same variable

```
/* module1.c */  
  
extern int variable;  
  
void increment(void) {  
    variable++;  
}
```

```
/* module2.c */  
  
#include  
  
int variable;  
  
void increment(void);  
  
int main(void) {  
    variable = 0;  
    increment();  
    printf("%d",variable);  
    return 0;  
}
```

The program emits `1` as its result. Can you explain why?

The program also emits `1` because the declaration of the `increment` function in `module2` is implicitly marked as having external linkage, i.e. it can be defined somewhere else.

A local function variable declared as static

```
static int counter;
```

A variable declared inside a function with the **static** attribute is a **static variable** (to distinguish it from an **automatic variable**). If a variable declaration is preceded by the static specifier, it means that the variable is brought to life **when the program starts and is destroyed when the program finishes its work**

A variable declared as a register

```
register int time;
```

As you probably know, both the program code and its data are stored inside the computer's memory. The processor accesses the memory whenever it has to get the next instruction and when it needs to read or write any data value. However, each processor has its own small, independent memory. It's coupled with the rest of the CPU inside the same chip and allows the data to be transferred several times faster than from/to the conventional memory. These integrated memory areas are divided between the **processor registers**.

The number of registers is very limited. They're used to perform arithmetic and logical operations, for the temporary storage of intermediate results and for manipulating the pointers. Normally, the developer isn't even aware of the use of the processor registers, because the compiler decides independently how to manage them.

Sometimes, however, the developer may want to let the compiler know that some of the variables are used intensively and therefore should not be stored in the conventional memory, but instead in registers. In some cases, it may cause the program to execute faster.

Despite its advantages, declaring a variable as a **register** has its limits. Here are some of the restrictions:

- a specific compiler may refuse to treat a variable as a register if its size exceeds a specific number of bits; the reason is that, should the processor use 64-bit registers, there is no physical possibility of storing a large array in there;
- a compiler may accept only the first few register variable declarations and may silently ignore all subsequent declarations; this is because the number of registers is limited and not all of them can be freely used;
- a variable declared as a register cannot be used as an argument to the unary **&** operator: this is a simple consequence of the fact that such a variable has no address.

So far, whenever we've mentioned pointers, they've been pointing toward data. The "C" language also allows us to declare and use **pointers to functions**.

The type of such a pointer determines the type of the function whose address the pointer is able to hold.

Take a look at the following declaration:

```
float (*ptr)(float);
```

Despite its advantages, declaring a variable as a **register** has its limits. Here are some of the restrictions:

It declares a variable named **ptr**, designed to **point to the functions** (not to all of them, but only to those which have one parameter of type **float**) and return a result of type **float**

Let's consider a scenario where we aim to create a function capable of printing a value table for any single-argument function. This function will generate values for the argument ranging from 0 to 1, with a step size of 0.01.

Below is a sample function:

```
1 void printlookup(float (*fun)(float)) {  
2     float x, fx;  
3  
4     for(x = 0.0; x <= 1.0; x += 0.01) {  
5         fx = fun(x);  
6         printf("%10.3f - %10.3f\n", x, fx);  
7     }  
8 }
```